

KLINIK MAKMUR JAYA

Arsitektur dan Infrastruktur

Dokumentasi Teknis Sistem Klinik Makmur Jaya

Versi Dokumen: 1.0

Tanggal: Juni 2026

Klasifikasi: Internal / Teknis

1. Pendahuluan

Dokumen ini menjelaskan arsitektur teknis dan infrastruktur sistem e-commerce Klinik Makmur Jaya. Tujuannya adalah memberikan gambaran menyeluruh mengenai komponen-komponen sistem, topologi jaringan yang digunakan, spesifikasi server yang direkomendasikan, serta kondisi implementasi saat ini dibandingkan dengan rancangan yang direkomendasikan untuk lingkungan production.

Seluruh status implementasi pada dokumen ini mengacu langsung pada kondisi source code dan konfigurasi project yang tersedia di repository. Komponen yang belum diimplementasikan dicantumkan sebagai rekomendasi untuk pengembangan berikutnya, bukan sebagai klaim ketersediaan fitur.

2. Ringkasan Status Implementasi

Tabel berikut merangkum status implementasi dari seluruh kebutuhan infrastruktur dan arsitektur sistem. Status ditandai dengan Selesai untuk komponen yang sudah berjalan, Sebagian untuk komponen yang sebagian tersedia atau masih dalam keterbatasan tertentu, dan Belum untuk komponen yang belum diimplementasikan pada versi saat ini.

Komponen	Status	Kondisi Saat Ini	Catatan
Frontend React SPA	Selesai	React 19, Vite 8, React Router DOM 7 via frontend/package.json	Perlu di-serve oleh web server/static hosting di production
Backend Laravel API	Selesai	Laravel 13, route API tersedia di routes/api.php	Perlu hardening konfigurasi untuk production
Database PostgreSQL	Selesai	DB_CONNECTION=pgsql di backend/.env.example	Backup, monitoring, dan tuning index perlu disiapkan
Autentikasi Token API	Selesai	Laravel Sanctum; endpoint protected via auth:sanctum	Token timeout 120 menit via SANCTUM_EXPIRATION
Queue Job Database	Sebagian	QUEUE_CONNECTION=database; job import dan laporan besar tersedia	Worker production dan monitoring queue belum dikonfigurasi
Web Server Production	Belum ada	Tidak ada konfigurasi Nginx/Apache/Caddy di repo	Direkomendasikan sebagai langkah deployment berikutnya
Load Balancer	Belum ada	Tidak ada konfigurasi load balancer di source code	Direkomendasikan jika traffic meningkat signifikan
WebSocket Real-Time	Belum ada	BROADCAST_CONNECTION=log; notifikasi via polling API	Direkomendasikan menggunakan broadcasting/service realtime
Redis Cache Production	Sebagian	Konfigurasi Redis ada di .env.example, cache default masih database	Redis belum aktif sebagai cache/queue production
Backup & Failover	Belum ada	Tidak ada script/konfigurasi backup di repo	Perlu SOP backup database, restore test, dan failover

3. Topologi Sistem

3.1 Kondisi Saat Ini

Pada kondisi saat ini, project disiapkan dan dijalankan sebagai aplikasi lokal atau development environment. Frontend berjalan sebagai React Single Page Application (SPA) melalui Vite, sementara backend berjalan sebagai Laravel API. Database menggunakan PostgreSQL sesuai konfigurasi di berkas .env.example.

Autentikasi API menggunakan Laravel Sanctum personal access token yang dikirimkan melalui header Authorization: Bearer <token>. Queue menggunakan driver database, dengan job yang tersedia untuk keperluan import data obat dan generate laporan berukuran besar.

3.2 Rekomendasi Topologi Production

Untuk lingkungan production, direkomendasikan agar arsitektur sistem dipisahkan menjadi beberapa lapisan yang masing-masing memiliki tanggung jawab berbeda. Lapisan paling luar adalah DNS dan terminasi TLS, yang mengarahkan trafik ke load balancer. Load balancer kemudian mendistribusikan permintaan ke web server untuk melayani aset statis React build, serta ke API server Laravel untuk keperluan logika bisnis.

Di sisi backend, API server terhubung ke server database PostgreSQL, queue worker Laravel untuk pemrosesan pekerjaan asinkron, storage lokal atau S3-compatible untuk menyimpan file upload resep, gambar obat, dan laporan, serta SMTP server untuk pengiriman email verifikasi pelanggan.

3.3 Jalur Trafik

Pengguna mengakses sistem melalui browser dengan membuka React SPA yang di-serve dari web server atau static hosting. Selanjutnya, frontend memanggil endpoint backend melalui library Axios. Endpoint publik seperti katalog obat, detail obat, kategori, login, register, dan verifikasi email dapat diakses tanpa autentikasi. Sementara itu, endpoint protected seperti cart, checkout, riwayat pesanan, notifikasi, dan seluruh fitur admin memerlukan token Bearer yang valid.

Backend mengakses database PostgreSQL untuk menyimpan dan mengambil data transaksi, stok, user, audit log, notifikasi, serta queue jobs. Untuk setiap proses registrasi pelanggan, backend mengirimkan email verifikasi melalui SMTP yang dikonfigurasi di berkas .env.

4. Komponen Utama Sistem

Sistem terdiri dari delapan komponen utama yang saling berinteraksi untuk membentuk satu kesatuan aplikasi e-commerce apotek. Komponen-komponen tersebut dirangkum dalam tabel berikut.

Komponen	Fungsi Utama	Status
React SPA	Antarmuka pengguna untuk pelanggan, admin, apoteker, dan kasir	Selesai
Laravel API	Business logic, validasi, autentikasi, manajemen pesanan, dan laporan	Selesai
PostgreSQL	Penyimpanan data utama: transaksi, stok, user, dan audit	Selesai

Komponen	Fungsi Utama	Status
Laravel Sanctum	Pengelolaan token API dan session timeout berbasis waktu	Selesai
Database Queue	Pemrosesan asinkron untuk import obat dan laporan besar	Sebagian
SMTP Server	Pengiriman email verifikasi akun pelanggan	Sebagian - perlu SMTP nyata
Load Balancer	Distribusi trafik ke beberapa instance API server	Belum diimplementasikan
WebSocket/Broadcasting	Notifikasi realtime tanpa mekanisme polling	Belum diimplementasikan

5. Spesifikasi Server yang Direkomendasikan

Spesifikasi berikut merupakan rekomendasi minimum dan rekomendasi awal untuk menjalankan sistem dalam lingkungan production dengan skala kecil hingga menengah, seperti apotek online atau klinik dengan volume transaksi harian yang moderat.

Komponen	Spesifikasi Minimum	Rekomendasi Awal	Catatan
API Server	2 vCPU, 4 GB RAM	4 vCPU, 8 GB RAM	Menjalankan PHP-FPM dan Laravel; queue worker lebih baik dipisah
Frontend / Static Hosting	1 vCPU, 1 GB RAM	CDN / Static Hosting	React build bersifat ringan; CDN meningkatkan latensi akses
Database PostgreSQL	2 vCPU, 4 GB RAM	4 vCPU, 8-16 GB RAM	Prioritaskan storage cepat (NVMe SSD) dan layanan backup rutin
Storage	50 GB SSD	100 GB SSD atau lebih	Untuk gambar obat, file resep, PDF laporan, dan log aplikasi
Bandwidth Jaringan	50 Mbps	100 Mbps atau lebih	Sesuaikan dengan jumlah pengguna aktif dan ukuran aset
Queue Worker	1 vCPU, 2 GB RAM	2 vCPU, 4 GB RAM	Khusus untuk pemrosesan import obat dan generate laporan besar

6. Redundansi, Failover, dan Backup

6.1 Redundansi dan Failover

Pada versi saat ini, mekanisme redundansi dan failover belum diimplementasikan dalam repository project. Sebagai rekomendasi untuk deployment production, disarankan agar server frontend, backend, database, dan queue worker dipisahkan menjadi instance yang independen. Penggunaan load balancer diperlukan apabila trafik meningkat dan diperlukan beberapa instance API server yang berjalan paralel.

Database disarankan menggunakan fitur backup harian dengan dukungan point-in-time recovery apabila tersedia dari penyedia layanan. File upload dan laporan sebaiknya disimpan pada storage yang memiliki

mekanisme backup sendiri. Health check secara berkala perlu dikonfigurasi untuk API, database, queue worker, dan SMTP server.

6.2 Backup Plan

Backup database PostgreSQL disarankan dilakukan setiap hari, begitu pula backup storage yang menyimpan file upload dan laporan. Retensi backup minimum yang direkomendasikan adalah 14 hingga 30 hari. Uji restore sebaiknya dilakukan secara berkala untuk memastikan integritas backup. Kepemilikan proses backup dan jadwal verifikasi perlu didokumentasikan secara jelas dan diketahui oleh seluruh pihak yang bertanggung jawab.

7. Keamanan Jaringan

7.1 Implementasi yang Sudah Tersedia

Beberapa mekanisme keamanan dasar sudah diimplementasikan dalam sistem. Seluruh endpoint yang memerlukan autentikasi dilindungi menggunakan Sanctum token. Middleware role tersedia untuk membatasi akses berdasarkan peran pengguna. Session dan token memiliki masa berlaku 120 menit yang dikonfigurasi melalui parameter `SANCTUM_EXPIRATION`. Audit log tersedia untuk mencatat banyak aktivitas penting di dalam sistem.

7.2 Rekomendasi untuk Production

Untuk lingkungan production, seluruh komunikasi antara klien dan server wajib menggunakan HTTPS. Port database sebaiknya dibatasi sehingga hanya dapat diakses dari IP API server yang sah. Firewall atau security group perlu diaktifkan untuk membatasi akses ke port yang tidak diperlukan. Kredensial di berkas `.env` harus dipisahkan dari repository dan tidak boleh di-commit ke version control. Disarankan pula untuk menambahkan Web Application Firewall (WAF) dan rate limiting pada endpoint-endpoint sensitif seperti login, register, dan checkout untuk mencegah penyalahgunaan.